

TinyML Gesture Reader

Gestusbaseret blackjack-simulation på Particle Photon 2

Kursus	ETTML-01 Tiny Machine Learning
Studerende	Erik Kjær Klint
Studienummer	201704536 / AU591167
Platform	Particle Photon 2 + ADXL343
Afleveringsdato	10. august 2026
Repository	ETTML—Gesture-Analysis

Semester-/eksamensprojekt

Indholdsfortegnelse

1	Resumé	3
2	Introduktion	3
3	Projektbeskrivelse og scope	3
4	Kravanalyse	4
4.1	Sporbarhed til kursuskrav	4
4.2	Funktionelle og ikke-funktionelle mål	4
5	Systemdesign	4
5.1	Dataflow og ansvar	4
5.2	Hardware og sensor	5
5.3	Driftsformer og protokol	5
6	Dataindsamling	5
6.1	Kalibrering og iterativ metode	5
6.2	Endeligt baseline-datasæt	6
7	Preprocessing og model	7
7.1	Featurekontrakt	7
7.2	MLP og eksport	8
8	Firmwareimplementering	8
8.1	Sampling og inference	8
8.2	LED-controller	8
9	Test og resultater	8
9.1	Offline evaluering	8
9.2	Embedded og LIVE-verifikation	9
9.3	Verifikationsstatus	9
10	Diskussion	10
11	Konklusion	10
12	Fremtidigt arbejde	10
13	Referencer	10
14	Appendiks A: Reproduktion	11
15	Appendiks B: Kodearkitektur og sporbarhed	11
16	Appendiks C: Datakontrakt	11
17	Appendiks D: Værktøjer og AI-assistance	11

1 Resumé

Projektet realiserer en komplet TinyML-kæde fra egen dataindsamling til lokal klassifikation på en Particle Photon 2. En ADXL343 måler acceleration på tre akser ved 400 Hz. Hvert firesekunders vindue omformes til 28 statistiske features og klassificeres af et multi-layer perceptron (MLP) med arkitekturen 28–32–16–5. De fem klasser er idle, ét, to og tre tryk samt venstre-højre-rystelse. Aktive klasser kortlægges til blackjack-kommandoerne stand, hit, exit og split.

Den endelige balancerede baseline består af 25 accepterede hardwareoptagelser, fem pr. klasse. Optagelsesprogrammet randomiserede instruktionerne og afviste automatisk forsøg med forkert antal impacts, bevægelse under idle eller utilstrækkelig shake. To forsøg blev afvist og gentaget. Alle accepterede filer har 1.600 samples, monotone timestamps, ingen manglende værdier og ingen sensor-clipping.

En reproducerbar stratificeret 80/20-split gav 80,0 % accuracy på fem testvinduer. Dette opfylder projektets demonstrationsmål, men testmængden er for lille til at dokumentere generel robusthed. På den endelige firmware blev model-inference observeret til 345 μ s i gennemsnit og 364 μ s maksimalt; cloud-buildet anvendte 27.950 B flash og 46.686 B RAM. LIVE blev verificeret fysisk: klassifikation udløste de specificerede RGB-mønstre. Projektet vurderes derfor som en fungerende, ressourcelet prototype med tydeligt dokumenterede begrænsninger.

2 Introduktion

TinyML kombinerer machine learning, signalbehandling og embedded udvikling. Kursets introduktion fremhæver lokal analyse på små 32-bit microcontrollere, datakvalitet, preprocessing og modeldeployment som centrale læringsmål [1]. Semesterprojektet kræver specifikt Photon 2, mindst én lokal sensor, egnet preprocessing, en ML-/DL-algoritme, egne mærkede data og et observerbart klasse-, regressions- eller anomalitetsoutput [2].

Gestusgenkendelse er egnet som klassifikationsproblem, fordi accelerationens form påvirkes af antal impacts, tempo, kraft, montage og sensororientering. En simpel tærskel kan registrere et kraftigt slag, men en robust femklasses beslutning kræver flere signalegenskaber. Projektets forskningsspørgsmål er derfor:

Kan en kompakt feature-baseret model på Photon 2 skelne idle, ét, to og tre tryk samt en lateral rystelse og omsætte resultatet til tydelig feedback med dokumenterbar latency og ressourcebrug?

Projektet er udført individuelt af Erik Kjær Klint. Arbejdet omfatter hardwareintegration, dataindsamling, Python-pipeline, modeltræning, C++-deployment, test og rapport. AI-assistance er beskrevet transparent i afsnittet om værktøjer; alle målinger og konklusioner er efterprøvet mod repository og hardware.

3 Projektbeskrivelse og scope

Artefaktet er en gestusgrænseflade til en blackjack-simulation, ikke en fuld spilmotor. Systemets ansvar slutter ved en valideret gestushændelse og en visuel/seriel kommando. Afgrænsningen holder fokus på kursets sensing–preprocessing–ML–deployment-kæde.

Klasse	Kommando	LIVE-feedback
idle	ingen	LED slukket
tap1	stand	blå i 1,0 s
tap2	hit	to blå pulser med 0,50 s startinterval
tap3	exit	tre røde pulser med ca. 0,33 s startinterval
shake_lr	split	rød–blå–rød–blå med 1,0 s startinterval

Firmware har tre eksplicite driftsformer. DEBUG giver sensordiagnostik uden gestushandlinger. TRAINING ejer en guided capture-state-machine. LIVE anvender den eksporterede model, confidence-gating, temporal smoo-

thing, impact-kontrol og LED-/serial-events. Denne adskillelse gør det muligt at forklare, teste og demonstrere samme sensor- og modelkontrakt med forskellig præsentation.

4 Kravanalyse

4.1 Sporbarhed til kursuskrav

Semesterbriefet angiver minimumssektionerne introduktion, projektbeskrivelse, kravanalyse, systemdesign, implementering, test/verifikation og konklusion samt et mål på omtrent 20 sider [2]. Tabellen viser, hvordan de tekniske krav spores til konkrete artefakter.

Krav	Implementering/evidens	Status
Photon 2 embedded hardware	Product/firmware/src/main.cpp; fysisk LIVE-test	Opfyldt
Lokal sensor	ADXL343 via I2C; DEVID- og read-error-kontrol	Opfyldt
Egne mærkede data	25 accepterede v3-CSV'er med labels og metadata	Opfyldt
Egnet preprocessing	mean removal, magnitude og 28 features	Opfyldt
ML-/DL-algoritme	StandardScaler + MLP 28-32-16-5	Opfyldt
Lokal forudsigelse	C++ forward-pass og softmax på Photon 2	Opfyldt
Observerbar tilstand	STATUS, EVENT og RGB-mønstre	Opfyldt
Delbar kode/data/dokumentation	repository med rå source, CSV og PDF	Klargjort
Hardwaredokumentation	pin-tabel, foto og officielle databladreferencer	Opfyldt i rapport

4.2 Funktionelle og ikke-funktionelle mål

De funktionelle mål er korrekt sampling, femklasse-inference, idle-afvisning, stabil event-emission og den specificerede LED-kortlægning. Projektets egne ikke-funktionelle targets er mindst 80 % held-out accuracy, under 50 ms ren model-inference, ingen dynamisk allokering i forward-pass og et firmwareimage, der passer komfortabelt på Photon 2. Disse er projekttargets og ikke tærskler fastsat af kurset.

Particle angiver en 200 MHz Arm Cortex-M33, op til 2 MB user-application og 3 MB RAM for Photon 2 [4]. Den platformsmargin gør et float32-MLP og en 1.600-sample ringbuffer realistisk, men kompakthed er stadig relevant for reproducerbar embedded praksis.

5 Systemdesign

5.1 Dataflow og ansvar

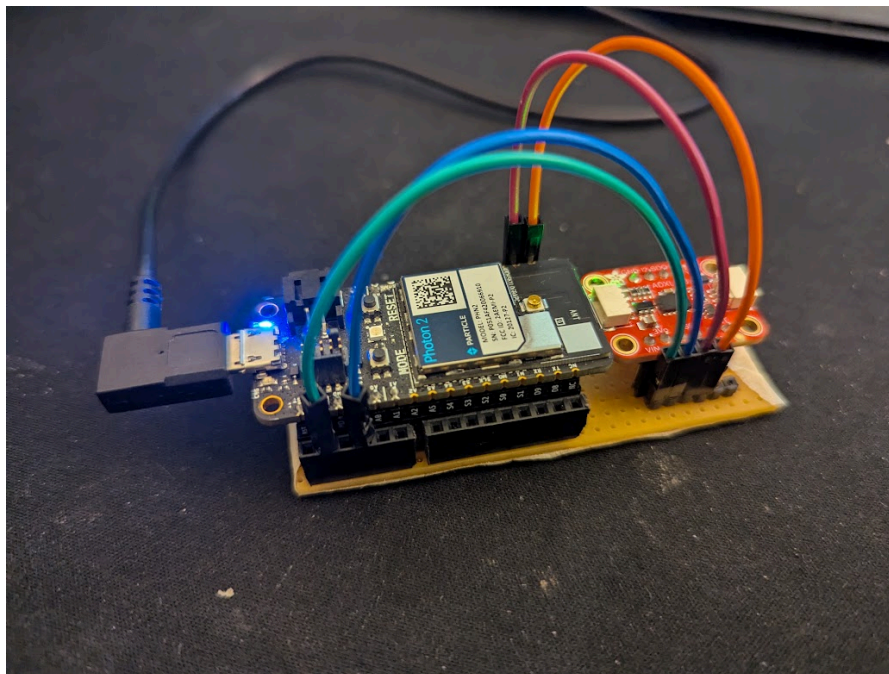
ADXL343400 Hz XYZ	→	4 s vindue 1.600 samples	→	28 features
		↓		
RGB + EVENT	←	Beslutningsfilter	←	Scaler + MLP

Tabel 1: End-to-end dataflow. Den samme featurekontrakt anvendes offline og i firmware.

Sensorlaget læser synkroniserede X/Y/Z-registre. Repræsentationslaget fjerner middelværdi pr. kanal, beregner magnitude og syv features for hver af fire kanaler. Inferencelaget standardiserer featurevektoren og udfører MLP-forward-pass. Beslutningslaget kræver tre konsekutive sikre resultater, anvender fire sekunders debounce og kontrollerer antal fysisk adskilte impacts for tap-klasser. Præsentationslaget ejer LED'en og serial-protokollen.

5.2 Hardware og sensor

ADXL343 understøtter $\pm 2/\pm 4/\pm 8/\pm 16$ g, full-resolution med nominelt 3,9 mg/LSB samt valgbare outputrater op til 3.200 Hz [3]. Projektet bruger ± 16 g full-resolution og 400 Hz ODR. Databladet oplyser desuden, at støjen stiger ved rater over 100 Hz; ved 400 Hz er typisk RMS-støj under ca. 1,5 LSB på X/Y og 2,2 LSB på Z [3]. Den højere rate blev valgt, fordi taps er korte impacts med efterfølgende mekanisk ringing, som blev undersamplet ved den tidligere 50 Hz-konfiguration.



Figur 1: Fysisk prototype med Photon 2, ADXL343 og lokal RGB-feedback.

Photon 2	ADXL343	Funktion
3V3	VCC	3,3 V forsyning
GND	GND	fælles reference
D0 / SDA	SDA	I2C-data
D1 / SCL	SCL	I2C-clock

Firmware prober adresserne $0x53$ og $0x1D$, forventer device-ID $0xE5$, konfigurerer measurement mode og rapporterer fejl via STATUS. Samples tidsstemples i mikrosekunder. I den endelige serie lå medianintervallet mellem 2.498 og $2.501 \mu\text{s}$; det svarer tæt til 400 Hz.

5.3 Driftsformer og protokol

Kommandoerne `MODE DEBUG`, `MODE TRAINING`, `MODE LIVE`, `STATUS` og `TAP_SCOPE` danner et lille tekstbaseret kontrolinterface. `LIVE` udsender kun stabile events som `EVENT, class=tap2, command=hit, score=...`. LED-sequenceren er non-blocking, så sampling og serial polling fortsætter under feedback.

6 Dataindsamling

6.1 Kalibrering og iterativ metode

Den første 50 Hz-baseline viste, at tætte tap-impacts kunne forsvinde mellem samples. En oscilloskoplignende diagnose blev derfor indført: 400 Hz synkron XYZ-capture, tre sekunders gul countdown, 0,5 sekunders pre-cue-baseline og 3,5 sekunder efter grøn GO-indikation. Eksperimentet viste også, at subjektive labels som “light”, “normal” og “firm” ikke havde rene, ikke-overlappende amplitudebånd. De bruges derfor til at skabe variation, ikke som ground-truth kraftklasser.

Det automatiske GUI randomiserer klasse, tempo og ønsket kraft. Det viser næste instruktion stort og centreret under gul ventetid, skifter til grøn under capture og viser den resterende tid. En konservativ validator kontrol-

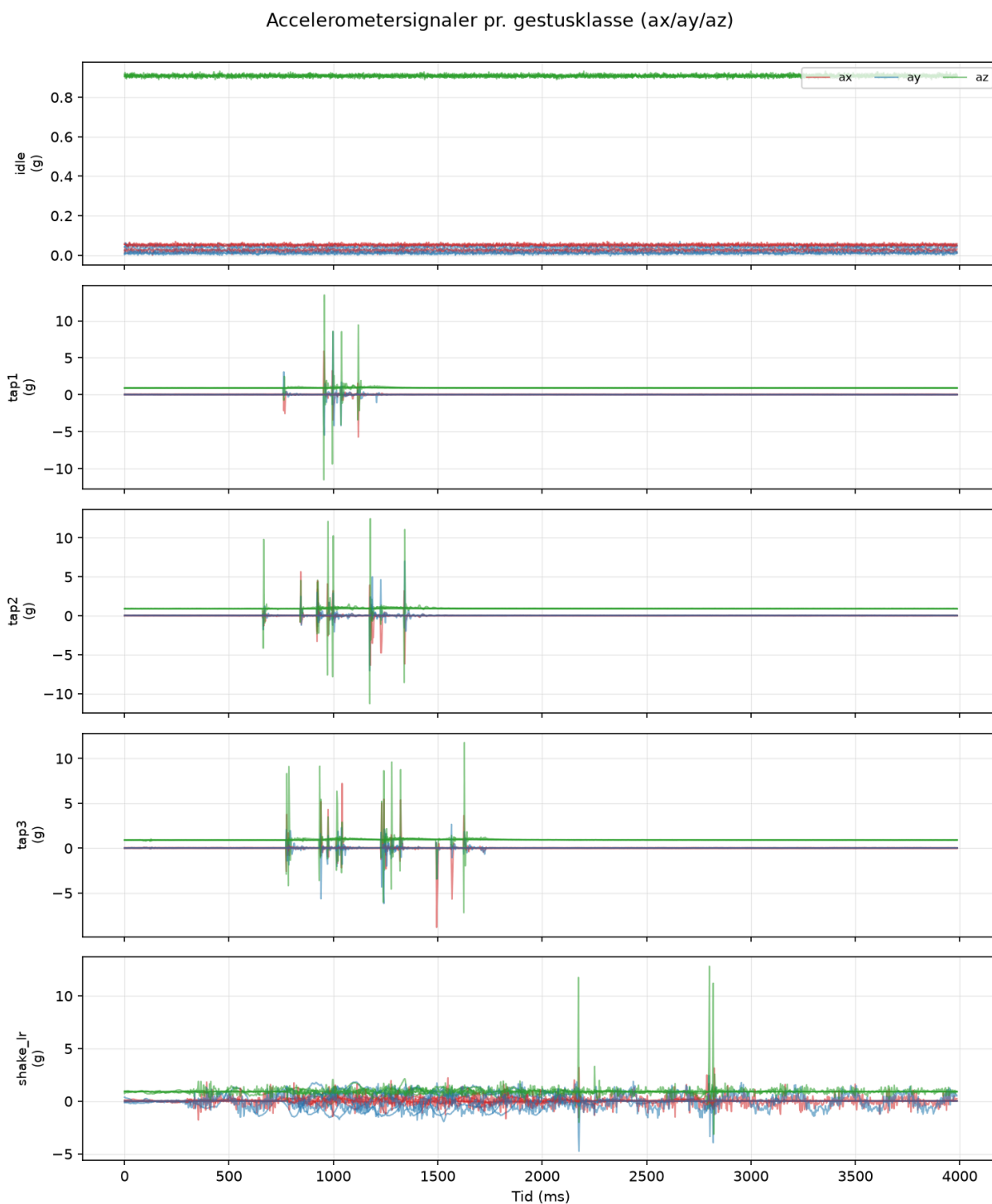
lerer clipping, idle-bevægelse, tap-event count og shake-varighed. Forkerte forsøg arkiveres som rejected og sættes tilbage i køen.

6.2 Endeligt baseline-datasæt

Den deployede session 20260810_141717 indeholder fem accepterede trials pr. klasse, i alt 25. Hver fil indeholder 1.600 samples plus label, requested pace/force, session-ID og acceptance-status. En JSON-sidecar gemmer bl.a. sampleinterval, peak, dynamisk RMS, aktivt tidsrum, event count og clipping-status; en PNG-sidecar giver visuel kontrol.

Klasse	Accepteret	Kvalitetskontrol
idle	5	0 events; lav dynamisk peak
tap1	5	præcis 1 impact-event
tap2	5	præcis 2 impact-events
tap3	5	præcis 3 impact-events
shake_lr	5	tilstrækkelig RMS og aktiv varighed
I alt	25	balanceret femklasse-baseline

To forsøg blev automatisk afvist: et idle-vindue havde 0,1248 g peak-axis og blev vurderet som bevæget; et tap1-forsøg indeholdt to impacts. Begge blev erstattet af valide trials. Alle 27 rå forsøg havde 1.600 rækker, monotone timestamps, ingen NaN og ingen clipping. Maksimalt isoleret timing-gap var 13,976 ms; medianen var fortsat korrekt. Det bør registreres som scheduler-jitter, men ændrede ikke event-integriteten.



Figur 2: Alle fem accepterede trials pr. klasse fra den deployede 400 Hz-session. Figuren står her sammen med datasetbeskrivelsen, ikke løserevet efter resultatafsnittet.

7 Preprocessing og model

7.1 Featurekontrakt

For hver kanal x beregnes først $x'_i = x_i - \bar{x}$. Magnitude er $m_i = (a_x^2 + a_y^2 + a_z^2)^{\frac{1}{2}}$. Derefter beregnes syv features for X, Y, Z og magnitude:

Feature	Betydning
standardafvigelse	typisk dynamisk spredning

minimum / maksimum	negative og positive ekstremer
range	maksimum minus minimum
energi	$\frac{1}{N} \sum_i (x'_i)^2$
peak count	lokale peaks over 0,05 g med otte-sample refractory
max abs diff	største absolutte forskel mellem nabosamples; impact-skarphed

Fire kanaler gange syv giver 28 features. Mean er bevidst ikke medtaget, fordi mean removal ellers ville gøre fire features næsten konstante. `StandardScaler` lærer middelværdi og standardafvigelse udelukkende fra træningsdelen og anvender $z = \frac{x-u}{s}$; denne adfærd følger bibliotekets dokumenterede kontrakt [5]. Samme scaler-konstanter eksporteres til C++.

7.2 MLP og eksport

Modellen er scikit-learn `MLPClassifier` med to ReLU-hidden layers på 32 og 16 neuroner samt femdimensionelt softmax-output. Scikit-learn beskriver `MLPClassifier` som en backpropagation-trænet ikke-lineær multiclass-model og bruger softmax til multiclass-output [6]. Konfigurationen bruger learning rate 0,001, random state 42 og højst 500 iterationer.

`export_model.py` træner og serialiserer scaler, label encoder og MLP-vægte. Eksporten er fail-closed: en træningsfejl kan ikke stille og roligt overskrive fungerende firmware med placeholder-vægte. Header/source/artefakter stages og erstattes atomisk efter succes. På enheden ligger parametrene som `const float-arrays`, og forward-pass bruger statiske aktiveringsbuffer uden heap-allokering.

8 Firmwareimplementering

8.1 Sampling og inference

LIVE bruger en 1.600-sample buffer ved 2.500 μ s targetinterval. Efter første fulde firesekunders vindue beholdes de nyeste 1.500 samples, og 100 nye samples giver 0,25 sekunders stride. Featureberegning og MLP-inference er samme numeriske rækkefølge som Python: magnitude, window mean removal, features, `StandardScaler`, `dense/ReLU/dense/ReLU/dense/softmax`.

Beslutningslaget adskiller sandsynlighed fra handling. En score under 0,75 behandles som idle. Tre ens sikre vinduer kræves før event. Fordi pilotens svageste skel var mellem nærliggende tap-klasser, anvendes den samme fysiske impact-envelope som i capture-validatoren som guard, efter MLP'en har identificeret tap-familien. Det er en hybrid sikkerhedsmekanisme: ML skelner signaltypen, mens kendt tidsstruktur forhindrer, at to målte impacts vises som ét. Denne brug af en regel skal fremgå åbent, da kursuskravet siger, at hovedopgaven ikke må reduceres til ren regelprogrammering [2].

8.2 LED-controller

LED-controlleren har én ejer og kører uden `delay()`. Den gemmer farve, alternativ farve, on/off-tider og antal pulser. `flashStep()` avancerer mønstret ud fra `millis()`. Det giver den observerbare outputkanal, som semesterbriefet kræver, uden at blokere 400 Hz acquisition.

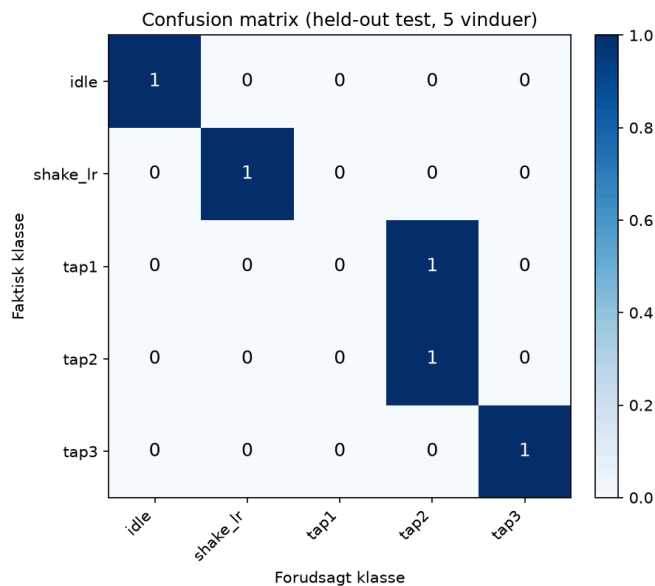
9 Test og resultater

9.1 Offline evaluering

Den reproducerbare evaluering bruger en stratificeret 80/20-split med random state 42. Det betyder 20 træningsvinduer og kun fem testvinduer—ét pr. klasse. Resultatet er derfor en smoke-test af kæden, ikke et præcist estimat af generalisering.

Metrik	Resultat	Vurdering
Accuracy	80,0 %	projekt mål nået
Macro precision	70,0 %	under mål
Macro recall	80,0 %	mål nået

Macro F1	73,3 %	under mål
Testvinduer	5	meget lille



Figur 3: Confusion matrix for den aktuelle 80/20-split. Fire af fem testvinduer er korrekte; tap1 forveksles med tap2.

Idle, shake, tap2 og tap3 blev korrekte; tap1 blev forudsagt som tap2. Ét eksempel svarer til 20 procentpoint, så 80 % må ikke beskrives som robust performance. En supplerende fem-fold cross-validation på de 25 vinduer gav 76 % samlet accuracy; idle og shake var stabile, mens tap-klasserne stod for fejlene. Denne sekundære analyse understøtter samme konklusion.

9.2 Embedded og LIVE-verifikation

Particle cloud compile af det endelige image lykkedes. Build-output og efterfølgende STATUS gav:

Måling	Resultat	Status
Firmware flash	27.950 B	passer
Firmware RAM	46.686 B	passer
Inference mean	345 μ s	< 50 ms
Inference max	364 μ s	< 50 ms
Sensor read errors	0	pass
Mode	LIVE	pass

Efter cloud flash rapporterede enheden MODE, current=LIVE, sensor=ok og nul read errors. En fysisk hands-on-test bekræftede, at en registreret gestus udløste LED-feedback; brugeren bekræftede direkte, at LIVE-kørslen fungerede. Den verificerer den komplette vertikale kæde. Der blev ikke udført en blind, randomiseret live-confusion-test med et på forhånd fast antal forsøg, så "live accuracy" rapporteres ikke som et tal.

9.3 Verifikationsstatus

Test	Status	Evidens/begrænsning
CSV-integritet og labels	Pass	27/27 rå forsøg auditeret
Samplingkadence	Pass	median 2.498–2.501 μ s
Offline femklasse-inference	Pass	80 % på 5 cases
Compile/flash	Pass	cloud build + flash success
On-device latency	Pass	345/364 μ s mean/max

LED-mapping	Pass	fysisk LIVE-kørsel
Lang idle false-trigger-test	Ikke målt	kræver tidsbestemt protokol
Flere brugere/session-held-out	Ikke målt	kun én operatør/session
Power/endurance	Ikke målt	uden for dagens slutttest

10 Diskussion

Projektets stærkeste resultat er ikke 80 %-tallet isoleret, men den verificerede data-til-firmware-kæde. Sampling, labeling, featureberegning, scaler, MLP, C++-forward-pass, beslutningslag og feedback er alle konkrete og observerbare. Den korte inference-tid viser, at den valgte model er langt under platformens compute-loft.

Den største trussel mod validitet er datasættets størrelse og struktur. Fem trials pr. klasse fra én bruger og én tæt session beskriver ikke variation mellem dage, montage eller personer. Testsplitten deler samme session mellem train og test og kan derfor overvurdere generalisering. En stærkere evaluering ville indsamle mindst tre uafhængige sessioner og holde en hel session ude.

400 Hz forbedrer synligheden af impacts, men firesekunders buffer giver højere opstartstid og holder en gestus i vinduet længe. Debounce forhindrer gentagne events, men sænker maksimal interaktionsrate. En fremtidig event-triggered 1–2 sekunders model kunne reducere latency uden at miste impact-forløbet.

Hybrid-guard'en er pragmatisk og fagligt forsvarlig, når den beskrives korrekt: MLP'en løser den ikke-lineære femklasse-repræsentation, mens event count anvendes som sikkerhedsconstraint på en kendt ordinal delopgave. Hvis guard'en alene kunne løse hele opgaven, ville det stride mod projektets ML-formål; den kan imidlertid ikke pålideligt skelne idle, tap og lateral shake under bred variation.

11 Konklusion

Projektet opfylder kursets centrale artefaktkrav: Photon 2, lokal ADXL343-sensor, egne labeled data, preprocessing, en trænet multiclass-MLP, lokal inference og observerbart serial-/LED-output. Den endelige baseline indeholder 25 balancerede accepterede 400 Hz-vinduer. Offline-splitten nåede 80 % accuracy, firmware brugte 27.950 B flash og 46.686 B RAM, og den rene inference tog omkring 0,35 ms. LIVE blev kompileret, flashet og fysisk verificeret.

Resultatet er en fungerende prototype, ikke en dokumentation af brugeruafhængig robusthed. Den korrekte faglige konklusion er derfor, at systemets arkitektur og deployment er bevist, mens generalisering, false-trigger-rate og længerevarende stabilitet kræver et større session-adskilt datasæt og en formaliseret live-test.

12 Fremtidigt arbejde

Prioriteret videre arbejde er: (1) flere sessioner og brugere, (2) session-held-out evaluering, (3) tidsbestemt idle false-trigger-test, (4) golden-vector parity-test mellem Python og C++, (5) kortere event-triggered vinduer og (6) først derefter BLE-integration eller sensorfusion. Kvantisering er ikke nødvendig for at passe på Photon 2, men kan bruges som et kursusrelevant optimeringseksperiment.

13 Referencer

- [1] Aarhus Universitet. (2026). **Welcome to TinyML—ETTML-01 Tiny Machine Learning** [kursusmateriale]. Brightspace, lokal PDF-kopi.
- [2] Aarhus Universitet. (2026). **Semester/Exam Project—ETTML-01 Tiny Machine Learning** [projektbrief]. Brightspace, lokal PDF-kopi.
- [3] Analog Devices. (2021). **ADXL343: 3-axis, ± 2 g/ ± 4 g/ ± 8 g/ ± 16 g digital accelerometer** (Rev. A). [Datablad](#).
- [4] Particle Industries. (2026). **Photon 2 datasheet**. [Particle documentation](#).
- [5] Scikit-learn Developers. (2026). **StandardScaler**. [API documentation](#).
- [6] Scikit-learn Developers. (2026). **Neural network models (supervised): MLP classification**. [User guide](#).
- [7] Géron, A. (2022). **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow** (3. udg.). O'Reilly Media.
- [8] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. **Journal of Machine Learning Research**, **12**, 2825–2830.
- [9] Raschka, S., Liu, Y., & Mirjalili, V. (2022). **Machine Learning with PyTorch and Scikit-Learn**. Packt.

14 Appendiks A: Reproduktion

Kommandoerne køres fra repository-rod:

```
source .venv/bin/activate
python Product/ml/randomized_capture_gui.py --port /dev/ttyACM1 --series 5
python Product/ml/train.py --config Product/ml/config.yaml
python Product/ml/export_model.py --config Product/ml/config.yaml
python Product/ml/generate_figures.py
particle compile photon2 Product/firmware --saveTo firmware.bin
particle flash TinyML_Node1 firmware.bin
```

LIVE aktiveres med MODE LIVE; STATUS kontrollerer mode, sensor, buffer, inference-count/timing og read errors.

15 Appendiks B: Kodearkitektur og sporbarhed

Fil/modul	Ansvar og vigtig invariant	Rapportafsnit
firmware/src/main.cpp	I2C, 400 Hz buffer, features, modes, decision-filter, LED. Sampling må ikke blokkes af feedback.	4, 6, 7
firmware/src/model_data.*	Genererede scaler-/MLP-konstanter og float32-forward-pass. Feature count skal være 28.	6
ml/randomized_capture_gui.py	Randomiseret GUI, serial capture, kvalitetstjek, retry og sidecars.	5
ml/train.py	Schema-normalisering, vinduer, features, split og metrikker.	6, 7
ml/export_model.py	Fail-closed atomisk eksport til C++.	6
ml/generate_figures.py	Figurer genberegnes fra aktuelle data og split.	5, 7
ml/config.yaml	Enkelt konfigurationspunkt for labels, sampling, model og decision logic.	5–7

Den centrale firmwaresekvens kan læses som følgende pseudokode:

```
loop:
    parse serial command
    if TRAINING: run capture state machine
    advance non-blocking LED
    if sample due: read synchronized XYZ into ringbuffer
    if window ready:
        features = stat_v2(window)
        scores = scaler + MLP + softmax(features)
        candidate = confidence/smoothing(scores)
        candidate = impact guard(candidate, window)
        if stable LIVE event: emit serial + LED pattern
```

Kommentarer i source forklarer kontrakter og begrundelser—ikke blot hvad hver C++-linje gør. Genereret model_data.cpp kommenteres ved generatoren, så man undgår håndredigering af modelparametre.

16 Appendiks C: Datakontrakt

CSV-felterne er time_us, x_g, y_g, z_g, afledt tid/delta/magnitude, label, requested pace/force, session_id og operator_accepted. En accepteret fil har præcis 1.600 samples. JSON-sidecars er audit metadata; PNG-sidecars er visuel evidens. Rejected-filer må ikke indgå i træning. Configens raw_dir peger eksplicit på den accepterede deploy-session, så gamle eller diagnostiske data ikke blandes ind implicit.

17 Appendiks D: Værktøjer og AI-assistance

Python 3.12, NumPy, pandas, scikit-learn, PyYAML, pyserial og Matplotlib blev brugt til data- og ML-pipelinen. Particle CLI/Device OS blev brugt til build og flash; Typst blev brugt til PDF. Codex og tidligere BlackBox AI blev anvendt til kode-/tekstassistance, fejlsøgning og strukturering. AI-output blev ikke behandlet som måledata eller faglig kilde; claims er kontrolleret mod source, datasæt, build-output, hardwarestatus og de anførte primærkilder. MLPClassifier og StandardScaler er softwarekomponenter, ikke AI-assistenter.